Chapter 7: Exploring Code with Language Libraries

Download this chapter as a PDF for offline reading:
Chapter-07-Exploring-Code-with-Language-Libraries.pdf

The QL language is powerful on its own, but real analysis happens through the standard libraries that model each programming language. This chapter shows how to use those libraries to explore a codebase and how to refine a query until it answers the question you actually care about.

Standard Libraries

Every supported language ships with a standard library that models its abstract syntax tree (AST), control flow, type system, and (for most languages) data flow. You import the library with a single statement:

```
import javascript
```

(The same pattern works for `python`, `java`, `cpp`, `csharp`, `go`, and the other supported languages.)

Once imported, you gain access to classes that represent program elements and to predicates that answer common questions about them.

Core classes (JavaScript example)

Class	What it represents
File	A source file
Function	A function or method
ArrowFunctionExpr	An arrow function
ClassDefinition	A class declaration
CallExpr	A function or method call
MemberAccess	A property access (obj.prop)
Expr / Stmt	Any expression or statement
StringLiteral	A string literal
DataFlow::Node	A node in the data-flow graph
RemoteFlowSource	A typical remote (attacker-controlled) source

Useful predicates

- `Function.getName()` — the function's name
- `Function.getParameter(int)` — a parameter by index
- `Function.getACall()` — a call that appears inside the function

- `CallExpr.getCalleeName()` — the name of the called function (when it is a simple name)
- `CallExpr.getArgument(int)` — an argument by index
- `Expr.getParent()` — the enclosing expression or statement

These are the building blocks you will use for almost every exploration query.

Exploration and Refinement

The usual workflow is:

1. Write a broad query that finds every instance of something interesting.
2. Run it and look at the results.
3. Add filters (or extract predicates) until the results match the question you intended to ask.

Starting broad

A minimal exploration query that lists every function:

```
/**
 * @name List functions
 * @description Lists every function and its name.
 * @kind problem
 * @id mustlearn/js-list-functions
 * @tags maintainability
 * @problem.severity recommendation
 * @precision high
 */
import javascript

from Function f
select f, "Function: " + f.getName()
```

(See `examples/chapter07/list-functions.ql`.)

Another useful starting point is "find every call to a particular function":

```
/**
 * @name Calls to eval
 * @description Finds calls to the eval function.
 * @kind problem
 * @id mustlearn/js-calls-to-eval
 * @tags security
 * @problem.severity warning
 * @precision high
 */
import javascript

from CallExpr c
```

```
where c.getCalleeName() = "eval"
select c, "Call to eval."
```

(See `examples/chapter07/calls-to-eval.ql`.)

Refining the query

The empty-functions query from Chapter 6 is a good candidate for refinement. Constructors, getters, and setters are often intentionally empty; we can exclude them with library predicates:

```
/**
 * @name Empty functions (refined)
 * @description Finds empty functions that are not constructors or getters/setters.
 * @kind problem
 * @id mustlearn/js-empty-functions-refined
 * @tags maintainability
 * @problem.severity recommendation
 * @precision high
 */
import javascript

from Function f
where
  not exists(f.getAStmt()) and
  not f.isConstructor() and
  not f.isGetter() and
  not f.isSetter()
select f, "Empty function: " + f.getName()
```

(See `examples/chapter07/empty-functions-refined.ql`.)

Common refinement techniques:

- Restrict by name, kind, or location (`f.getFile().getRelativePath().matches("%test%")`).
- Exclude synthetic or compiler-generated elements.
- Require a particular structural relationship (`exists(CallExpr c | c.getEnclosingFunction() = f and …)`).
- Extract a predicate once the same filter appears in more than one query (as shown in Chapter 5).

Interactive exploration

In the CodeQL extension for VS Code you can:

- Use the query console or “CodeQL: Run Query” to iterate quickly.
- Jump from a result to the corresponding source location.
- Hover on library classes and predicates to read their documentation.
- Use the “CodeQL: Quick Query” command to experiment with small snippets against the current database.

These tools turn the standard libraries into an interactive map of the codebase.

Next Steps

You can now navigate a codebase with the language libraries and refine a query until it produces the results you want. The next chapter introduces data-flow analysis—the technique that lets you track values from sources to sinks instead of only matching syntactic patterns.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter07/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL